

script.js

```
// =====
// 1. 전역 상태 머신 관리 (State Machine)
// =====
const state = {
  currentScreen: 'start', // 'start', 'menu', 'game'
  activeLesson: null,      // 1: Train, 2: Toy, 3: Rocket
  activeNode: 1,          // 1, 2, 3 단계 노드
  ccqActive: false,
  score: 0
};

// 캔버스 엘리먼트 셋업
const canvas = document.getElementById('gameCanvas');
const ctx = canvas.getContext('2d');

// =====
// 2. 오디오 인프라 설정 (Web Audio API)
// =====
let audioCtx = null;
let masterGain = null;
let synthesizerNode = null; // 지속형 사운드 오실레이터
let filterNode = null;      // 오디오 모폴로지용 로우패스 필터

function initAudio() {
  if (audioCtx) return;

  // 브라우저 호환성 및 컨텍스트 바인딩
  const AudioContextClass = window.AudioContext || window.webkitAudioContext;
  audioCtx = new AudioContextClass();

  // 마스터 볼륨 게인 생성 및 연결
  masterGain = audioCtx.createGain();
  masterGain.gain.setValueAtTime(0.15, audioCtx.currentTime);
  masterGain.connect(audioCtx.destination);

  // 오디오 모폴로지 제어용 핵심 필터 생성
  filterNode = audioCtx.createBiquadFilter();
  filterNode.type = 'lowpass';
  filterNode.frequency.setValueAtTime(20000, audioCtx.currentTime); // 초기값은 완전 개방
  filterNode.connect(masterGain);
}

// 오디오 주파수 필터 감쇠 제어 (Muffle 효과)
function applyFilterMuffle(targetFreq, duration) {
  if (!audioCtx || !filterNode) return;
  filterNode.frequency.exponentialRampToValueAtTime(targetFreq, audioCtx.currentTime + duration);
}

// 오디오 사운드 합성 회로 시작
function startSynthSound(type) {
  stopSynthSound();
  if (!audioCtx) return;

  synthesizerNode = audioCtx.createOscillator();

  if (type === 'train') {
    synthesizerNode.type = 'sawtooth';
    synthesizerNode.frequency.setValueAtTime(65, audioCtx.currentTime); // 목직한 저음 구동음
  } else if (type === 'toy') {
    synthesizerNode.type = 'triangle';
    synthesizerNode.frequency.setValueAtTime(440, audioCtx.currentTime); // 가벼운 전자음 멜로디 베이스
  } else if (type === 'rocket') {
    synthesizerNode.type = 'square';
    synthesizerNode.frequency.setValueAtTime(45, audioCtx.currentTime); // 초저주파 진동 굉음
  }

  synthesizerNode.connect(filterNode);
  synthesizerNode.start();
}

function stopSynthSound() {
  if (synthesizerNode) {
    try {
      synthesizerNode.stop();
      synthesizerNode.disconnect();
    } catch (e) {}
    synthesizerNode = null;
  }
}
```

```

}

// 짧은 효과음 생성 피드백 루틴
function playClickSound(freq, duration) {
  if (!audioCtx) return;
  const osc = audioCtx.createOscillator();
  const gain = audioCtx.createGain();
  osc.type = 'sine';
  osc.frequency.setValueAtTime(freq, audioCtx.currentTime);
  gain.gain.setValueAtTime(0.1, audioCtx.currentTime);
  gain.gain.exponentialRampToValueAtTime(0.01, audioCtx.currentTime + duration);
  osc.connect(gain);
  gain.connect(masterGain);
  osc.start();
  osc.stop(audioCtx.currentTime + duration);
}

// =====
// 3. 물리 지오메트리 개체 정의 (Physics & Graphics)
// =====
const train = { x: -120, y: 220, width: 140, height: 50, speed: 3.5 };
const platform = { edgeLine: 260 };
const smokeParticles = [];

const toyFactory = {
  blocks: [],
  isRunning: true,
  spawnTimer: 0
};

const rocket = { x: 210, y: 350, width: 40, height: 90, shake: 0, flamePulse: 0 };

// 팩토리 블록 초기화
function resetToyBlocks() {
  toyFactory.blocks = [];
  toyFactory.isRunning = true;
  for (let i = 0; i < 5; i++) {
    toyFactory.blocks.push({
      x: 80 + Math.random() * 260,
      y: Math.random() * 200,
      size: 30 + Math.random() * 20,
      color: ['#ff4757', '#eccc68', '#1e90ff', '#2ed573'][Math.floor(Math.random() * 4)]
    });
  }
}

// =====
// 4. 레슨별 시나리오 및 CCQ 데이터 명세 매트릭스
// =====
const lessonData = {
  1: {
    title: "🚂 Train Platform Adventure!",
    nodes: {
      1: { text: "The train is going bye-bye right now!", ccq: "Is the train all gone? 🙄", ans: "no" },
      2: { text: "The train just left!", ccq: "Can we get on the train right now? 🚂", ans: "no" },
      3: { text: "Another train comes at 8:30!", ccq: "Is the new train here right now? 🕒", ans: "no" }
    }
  },
  2: {
    title: "🧸 Magical Toy Factory!",
    nodes: {
      1: { text: "The toy blocks are falling down fast!", ccq: "Are the blocks still moving down? 🧸", ans: "yes" },
      2: { text: "The toy shop had closed before the teddy bears arrived.", ccq: "Is the toy shop open right now? 🏪", ans: "no" },
      3: { text: "The lights might flash if it gets too busy.", ccq: "Are the lights flashing right now? 💡", ans: "no" }
    }
  },
  3: {
    title: "🚀 Space Rocket Launchpad!",
    nodes: {
      1: { text: "The rocket is getting super hot right now!", ccq: "Is the rocket really hot right now? 🔥", ans: "yes" },
      2: { text: "It might zoom to the moon if we press this!", ccq: "Did it fly to the moon yet? 🌙", ans: "no" },
      3: { text: "Wow! Look at the big space field!", ccq: "Adventure is all complete! 🎉", ans: "yes" }
    }
  }
};

// =====
// 5. 프레임 바이 프레임 핵심 루프 (Game Loop)
// =====
function updatePhysics() {

```

```

if (state.currentScreen !== 'game') return;

// ---- LESSON 1: 기차역 플랫폼 물리 연산 ----
if (state.activeLesson === 1) {
  if (state.activeNode === 1) {
    train.x += train.speed;

    // 엄격한 픽셀 경계 판정: 시제 엣지(Tense Edge) 돌파 감지 트리거
    if (train.x > platform.edgeLine) {
      state.activeNode = 2;
      applyFilterMuffle(250, 0.4); // 로우패스 필터 주파수를 250Hz로 급격히 감쇠 (소리 먹먹해짐)
      triggerCCQ();
    }
  }

  // 연기 입자 시뮬레이션
  if (state.activeNode === 2) {
    if (Math.random() < 0.15) {
      smokeParticles.push({ x: platform.edgeLine + 40, y: 240, r: 10, alpha: 1.0 });
    }
  }
  for (let i = smokeParticles.length - 1; i >= 0; i--) {
    smokeParticles[i].y -= 1.2;
    smokeParticles[i].x += Math.sin(smokeParticles[i].y * 0.05) * 0.5;
    smokeParticles[i].alpha -= 0.01;
    if (smokeParticles[i].alpha <= 0) smokeParticles.splice(i, 1);
  }
}

// ---- LESSON 2: 마법 장난감 공장 물리 연산 ----
if (state.activeLesson === 2) {
  if (state.activeNode === 1 && toyFactory.isRunning) {
    toyFactory.blocks.forEach(b => {
      b.y += 2.5;
      if (b.y > 500) { b.y = -50; b.x = 80 + Math.random() * 260; }
    });

    toyFactory.spawnTimer++;
    if (toyFactory.spawnTimer > 180) { // 일정 시간 작동 후 다음 과거 잔상 노드로 자동 이전
      toyFactory.isRunning = false; // 루프 업데이트 중단으로 빠대 정지 연출 (Trace 형성)
      state.activeNode = 2;
      applyFilterMuffle(150, 0.2); // 소리 완전 단절 효과 변조
      triggerCCQ();
    }
  }
}

// ---- LESSON 3: 우주 로켓 발사대 물리 연산 ----
if (state.activeLesson === 3) {
  if (state.activeNode === 1) {
    rocket.shake = Math.sin(Date.now() * 0.08) * 4; // 강렬한 지오메트리 흔들림 연산
    rocket.flamePulse = 20 + Math.random() * 25;

    if (Math.random() < 0.004) { // 확률적 시제 엣지 돌파 감지
      state.activeNode = 2;
      rocket.shake = 0; // 흔들림 엄격 정지
      applyFilterMuffle(1200, 1.55); // 신비로운 우주 스위프 주파수 이행
      triggerCCQ();
    }
  }
}

function renderGraphics() {
  // 이전 프레임 깨끗하게 밀어내기 (clearRect 엄격 준수)
  ctx.clearRect(0, 0, canvas.width, canvas.height);

  // 1단계 배경 하늘 드로잉
  ctx.fillStyle = '#1a1a24';
  ctx.fillRect(0, 0, canvas.width, canvas.height);

  if (state.currentScreen !== 'game') return;

  // ---- LESSON 1 캔버스 직접 그리기 ----
  if (state.activeLesson === 1) {
    // 철로 드로잉
    ctx.fillStyle = '#7f8c8d';
    ctx.fillRect(0, 260, canvas.width, 15);

    // 플랫폼 노란 안전 경계선 (Tense Edge 마커)

```

```

ctx.fillStyle = '#ffcc00';
ctx.fillRect(platform.edgeLine, 260, 8, canvas.height - 260);

// Node 1: 움직이는 파란색 블록 기차 그리기
if (state.activeNode === 1) {
  ctx.fillStyle = '#1e90ff';
  ctx.fillRect(train.x, train.y, train.width, train.height);
  // 기차 바퀴 드로잉
  ctx.fillStyle = '#2c3e50';
  ctx.beginPath(); ctx.arc(train.x + 30, train.y + 50, 12, 0, Math.PI * 2); ctx.fill();
  ctx.beginPath(); ctx.arc(train.x + 110, train.y + 50, 12, 0, Math.PI * 2); ctx.fill();
}

// Node 2: 기차가 완전히 빠져나간 철로 뒤 잔상 연기 그리기
smokeParticles.forEach(p => {
  ctx.fillStyle = 'rgba(220, 225, 230, ' + p.alpha + ')';
  ctx.beginPath(); ctx.arc(p.x, p.y, p.r, 0, Math.PI * 2); ctx.fill();
});

// Node 3: 예정된 구조화 계층 - 점멸 전광판 및 시계 자리 배치
if (state.activeNode === 3) {
  ctx.save();
  ctx.shadowBlur = 15;
  ctx.shadowColor = '#ffcc00';
  ctx.fillStyle = (Math.floor(Date.now() / 300) % 2 === 0) ? '#ffcc00' : '#443300';
  ctx.beginPath(); ctx.arc(210, 150, 40, 0, Math.PI * 2); ctx.fill();
  ctx.restore();

  ctx.fillStyle = '#1a1a24';
  ctx.font = 'bold 16px sans-serif';
  ctx.fillText("8:30", 192, 155);
}
}

// --- LESSON 2 캔버스 직접 그리기 ---
if (state.activeLesson === 2) {
  // 장난감 상자 컨베이어 벨트 베이스 벨트 드로잉
  ctx.fillStyle = '#34495e';
  ctx.fillRect(50, 0, 320, canvas.height);

  // 공장 내부 떨어지는 다채로운 원시 블록 렌더링
  toyFactory.blocks.forEach(b => {
    ctx.fillStyle = b.color;
    ctx.fillRect(b.x, b.y, b.size, b.size);
  });

  // Node 2 완료 진상: 닫힘 표시 패널 강제 레이어 드로잉
  if (state.activeNode === 2) {
    ctx.fillStyle = 'rgba(231, 76, 60, 0.85)';
    ctx.fillRect(60, 180, 300, 80);
    ctx.fillStyle = 'ffffff';
    ctx.font = 'bold 24px sans-serif';
    ctx.fillText("🚪 CLOSED", 140, 230);
  }
}

// --- LESSON 3 캔버스 직접 그리기 ---
if (state.activeLesson === 3) {
  // 발사대 고정 빔 그리기
  ctx.fillStyle = '#7f8c8d';
  ctx.fillRect(180, 200, 10, 200);

  // Node 1: 엔진 하단 추진 불꽃 그리기 (랜덤 반지름)
  if (state.activeNode === 1) {
    ctx.fillStyle = '#ff4757';
    ctx.beginPath();
    ctx.arc(rocket.x + 20 + rocket.shake, rocket.y + 90, rocket.flamePulse, 0, Math.PI * 2);
    ctx.fill();
  }

  // 로켓 메인 바디 벡터 그래픽 그리기
  ctx.fillStyle = '#f1f2f6';
  ctx.fillRect(rocket.x + rocket.shake, rocket.y, rocket.width, rocket.height);
  // 로켓 꼬깔콘 머리 그리기
  ctx.fillStyle = '#ff6b81';
  ctx.beginPath();
  ctx.moveTo(rocket.x + rocket.shake, rocket.y);
  ctx.lineTo(rocket.x + 20 + rocket.shake, rocket.y - 30);
  ctx.lineTo(rocket.x + rocket.width + rocket.shake, rocket.y);
  ctx.fill();
}

```

```

// Node 2 가능성 조동사: 스위치 주변의 무작위 오파시티 아우라 그리기
if (state.activeNode === 2) {
  let pulseOpacity = 0.4 + Math.sin(Date.now() * 0.01) * 0.2;
  ctx.fillStyle = 'rgba(46, 204, 113, ' + pulseOpacity + ')';
  ctx.beginPath();
  ctx.arc(rocket.x + 20, rocket.y + 45, 50, 0, Math.PI * 2);
  ctx.fill();
}
}
function gameLoop() {
  updatePhysics();
  renderGraphics();
  requestAnimationFrame(gameLoop);
}
// =====
// 6. 상태 제어 및 인터랙션 액션 UI 핸들러
// =====
function updateUI() {
  // 전체 스크린 레이어 컨트롤 토글
  document.getElementById('start-overlay').classList.add('hidden');
  document.getElementById('menu-layer').classList.add('hidden');
  document.getElementById('game-layer').classList.add('hidden');
  if (state.currentScreen === 'start') {
    document.getElementById('start-overlay').classList.remove('hidden');
  } else if (state.currentScreen === 'menu') {
    document.getElementById('menu-layer').classList.remove('hidden');
    stopSynthSound();
  } else if (state.currentScreen === 'game') {
    document.getElementById('game-layer').classList.remove('hidden');
    const currentLessonConfig = lessonData[state.activeLesson];
    const currentNodeConfig = currentLessonConfig.nodes[state.activeNode];
    document.getElementById('board-text').innerText = currentNodeConfig.text;
  }
}
function triggerCCQ() {
  state.ccqActive = true;
  const currentLessonConfig = lessonData[state.activeLesson];
  const currentNodeConfig = currentLessonConfig.nodes[state.activeNode];
  document.getElementById('ccq-text').innerText = currentNodeConfig.ccq;
  document.getElementById('ccq-panel').classList.remove('hidden');
}
function handleCCQAnswer(userChoice) {
  if (!state.ccqActive) return;
  const currentLessonConfig = lessonData[state.activeLesson];
  const currentNodeConfig = currentLessonConfig.nodes[state.activeNode];
  if (userChoice === currentNodeConfig.ans) {
    // 정답 처리 매커니즘
    playClickSound(880, 0.15); // 경쾌한 정답음 피드백
    document.getElementById('board-text').innerText = "Right! Great job! 🌟";
    document.getElementById('ccq-panel').classList.add('hidden');
    state.ccqActive = false;
    // 다음 노드 체인 연결
    setTimeout(() => {
      if (state.activeNode < 3) {
        state.activeNode++;
        // 소리 주파수 필터 다시 리셋하여 다음 정황으로 오픈
        if (filterNode) filterNode.frequency.setValueAtTime(20000, audioCtx.currentTime);
        updateUI();
      } else {
        // 레슨 완료 피드백 후 메뉴 탈출
        document.getElementById('board-text').innerText = "Adventure Clear! Super! 🏆";
        setTimeout(() => {
          state.currentScreen = 'menu';
          updateUI();
        }, 2000);
      }
    }, 1500);
  } else {
    // 오답 피드백의 시각적 결합 유도 원칙
    playClickSound(220, 0.3); // 무거운 좌절 피드백음
    document.getElementById('board-text').innerText = "Look closely at the screen again! 🙄";
  }
}
// =====
// 7. 브라우저 이벤트 바인딩 및 가동 초기화
// =====
document.getElementById('btn-start').addEventListener('click', () => {
  initAudio(); // 오디오플레이 제약 완벽 우회 바인딩
  playClickSound(600, 0.1);
});

```

```
state.currentScreen = 'menu';
updateUI();
});
document.querySelectorAll('.btn-menu').forEach(btn => {
  btn.addEventListener('click', (e) => {
    initAudio();
    const targetLesson = parseInt(e.target.getAttribute('data-lesson'));
    state.currentScreen = 'game';
    state.activeLesson = targetLesson;
    state.activeNode = 1;
    state.ccqActive = false;
    document.getElementById('ccq-panel').classList.add('hidden');
    playClickSound(523.25, 0.12);
    // 테마별 물리 리셋 및 합성음 작동 시작
    if (targetLesson === 1) { train.x = -120; startSynthSound('train'); }
    if (targetLesson === 2) { resetToyBlocks(); startSynthSound('toy'); }
    if (targetLesson === 3) { startSynthSound('rocket'); }
    updateUI();
  });
});
document.getElementById('btn-back').addEventListener('click', () => {
  playClickSound(400, 0.08);
  state.currentScreen = 'menu';
  updateUI();
});
document.getElementById('ccq-opt-yes').addEventListener('click', () => handleCCQAnswer('yes'));
document.getElementById('ccq-opt-no').addEventListener('click', () => handleCCQAnswer('no'));
// 루프 구동 시작 (백그라운드에서 실시간 무한 대기 가동)
gameLoop();
```